

PySpark e Apache Kafka Para Processamento de Dados em Batch e Streaming

11 %

Trilha

Fórum

video

04:43

O Processo de Extração

video

06:05

O Processo de Transformação

video

07:47

O Processo de Carga

video

09:56

Python e Suas Bibliotecas Para ETL



ebook

01:56

Manipulação de Dados com Pandas



ebook

01:17

Trabalhando com JSON e XML em Python



ebook

01:04

Conexão com Bancos de Dados Usando SQLAlchemy



ebook

01:46

Automação de Pipelines de ETL

ebook

02:20

Logging e Monitoramento de ETLs

ebook

02:18



Data Science Academy

Conexão com Bancos de Dados Usando SQLAlchemy

Conectar-se a bancos de dados em Python pode ser facilitado significativamente usando a biblioteca SQLAlchemy.

SQLAlchemy é um toolkit SQL e Object-Relational Mapping (ORM) para Python. Ele oferece uma abordagem de alto nível e abstrata para interagir com bancos de dados SQL, permitindo que os desenvolvedores escrevam código Python em vez de SQL para criar, ler, atualizar e excluir dados.

Instalação

Para começar, você precisará instalar o SQLAlchemy, que pode ser feito usando pip:

```
1 pip install sqlalchemy
```

Conexão com o Banco de Dados

Para se conectar a um banco de dados, você primeiro precisa criar um 'engine'. O 'engine' é o ponto de partida para qualquer interação com o banco de dados no SQLAlchemy. A string de conexão varia de acordo com o tipo de banco de dados que você está usando (PostgreSQL, MySQL, SQLite, etc.).

Exemplo para SQLite:

```
1 from sqlalchemy import create_engine
2
3 # Conexão SQLite (banco de dados local)
4 engine = create_engine('sqlite:///meubanco.db')
5
```

Para outros bancos de dados, você precisará especificar mais informações na string de conexão, como nome de usuário, senha, host e banco de dados. Por exemplo, para PostgreSQL:

```
1 # Conexao PostgreSQL
2 engine = create_engine('postgresql+psycopg2://usuario:senha@host:port/banco')
3
4
```

Executando Consultas

Com o 'engine', você pode executar consultas SQL diretamente ou usar a ORM para interagir de forma mais "pythonica".

Usando Consultas SQL

```
1 with engine.connect() as connection:
2     result = connection.execute("SELECT * FROM minha_tabela")
3     for row in result:
4         print(row)
```

Usando a ORM

SQLAlchemy ORM permite que você defina classes Python que mapeiam para tabelas de banco de dados.

```
1 from sqlalchemy.ext.declarative import declarative_base
2 from sqlalchemy import Column, Integer, String
3 from sqlalchemy.orm import sessionmaker
4
5 Base = declarative_base()
6
7 # Definindo uma classe que mapeia para uma tabela do banco de dados
8 class Usuario(Base):
9     __tablename__ = 'usuarios'
10
11     id = Column(Integer, primary_key=True)
12     nome = Column(String)
13     idade = Column(Integer)
14
15 # Criar tabela
16 Base.metadata.create_all(engine)
17
18 # Criar uma sessão
19 Session = sessionmaker(bind=engine)
20 session = Session()
21
22 # Inserir um novo registro
23 novo_usuario = Usuario(nome="João", idade=30)
24 session.add(novo_usuario)
25 session.commit()
26
27 # Consultar registros
28 for usuario in session.query(Usuario).all():
29     print(usuario.nome, usuario.idade)
30
```

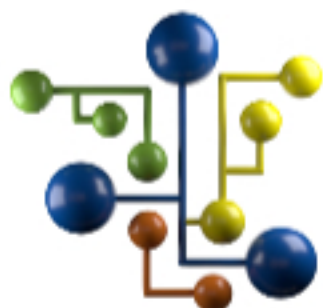
Considerações

* **Segurança:** Ao conectar-se a bancos de dados, especialmente em ambientes de produção, sempre considere aspectos de segurança, como armazenamento seguro de credenciais.

* **Drivers de Banco de Dados:** Dependendo do banco de dados que você está usando, pode ser necessário instalar um driver adicional (como psycopg2 para PostgreSQL).

* **Sessões e Transações:** Gerencie cuidadosamente sessões e transações, especialmente em operações de escrita, para manter a integridade dos dados.

SQLAlchemy é uma ferramenta poderosa que abstrai muitos dos detalhes de baixo nível do SQL, permitindo que você se concentre mais na lógica do seu aplicativo e menos na gestão do banco de dados.



Equipe DSA

Continue trilhando uma excelente jornada de aprendizagem.

